

软件需求规格说明书

1 范围

1.1 标识

系统名称：伴游 (TravelMate) 出行旅游平台

缩略词：TravelMate

版本号：V1.0

开发小组：Miracle 开发小组

1.2 系统概述

“伴游(TravelMate)”是一款面向出行旅游场景的一体化平台，旨在围绕“行前规划——行中服务——行后分享”的完整旅程为用户提供服务。系统主要功能包括：

资源预订：机票、火车票购买，酒店住宿及本地玩乐预订。

智能服务：基于大语言模型的AI行程规划及AI客服Agent。

社交分享：旅途社区游记分享与互动。

管理端：后台资源维护、内容安全审核及系统可观测性监控。

本项目为软件工程基础课程模拟项目，强调核心需求覆盖、功能可用性、小并发稳定性以及文档一致性。

1.3 文档概述

本文档描述了对“伴游”平台的CSCI需求及验收条件，作为系统设计与测试的基础。文档内容涵盖功能、性能、环境、安全性及外部接口需求，适用于开发团队、教师及助教。

1.4 基线

《软件开发计划书》V1.0

《TravelMate 核心需求描述文档》

《软件概要设计说明书》

《项目构思文档》

2 引用文件

《软件工程基础2026春大作业要求》

《软件开发计划书》V1.0(Miracle小组)

《软件详细设计说明书》(Miracle小组)

DeepSeek API官方技术文档

Spring Boot, Vue3, MySQL官方技术规范

3 需求

3.1 所需的状态和方式

系统需支持以下运行状态：

正常运行状态：用户可进行资源检索、AI行程生成、下单支付及社区发布。

离线/维护状态：后台进行资源批量更新或数据库维护。

降级模式：在大模型API响应超时，系统需提供本地Mock数据支持或错误重试提示。

3.2 需求概述

3.2.1 目标

核心目标：解决传统旅游平台信息破碎、行程规划繁琐的问题，利用AI技术实现结构化、个性化的旅程定制。
处理流程：用户输入需求->AI生成结构化行程->匹配本地Mock资源->下单预占库存->模拟支付->订单履约->行后社区分享。
系统定位：作为课程驱动的模拟软件，不接入真实支付与票务接口，采用“业务闭环真实、外部依赖模拟”策略。

3.2.2 运行环境

客户端：支持Chrome、Edge等主流Web浏览器。
服务端：Java 21, Spring Boot 3.x。
数据库：MySQL 8.0, Redis缓存。

3.2.3 用户的特点

普通游客：需要直观的资源搜索和一键式AI行程生成，对交互效率有要求。
内容创作者：使用社区发布工具进行多图上传和位置打卡。
平台管理员：进行资源管理、内容审核及系统运行状态实时监控。

3.2.4 关键点

AI Agent集成：利用DeepSeek API实现JSON结构化输出，通过结构化Prompt工程与多轮对话上下文管理实现个性化行程规划与智能客服问答。
库存防超卖：采用“Redis 缓存预减+数据库乐观锁”策略保证订票订房场景的并发安全。
状态机控制：严格管理订单从“待支付”到“已出行”的状态跃迁，防止状态乱序。

3.2.5 约束条件

开发期限：2026年4月至6月，共9周。
技术栈限制：必须采用Vue 3, Spring Boot 3.x, MySQL, Redis等指定栈
经费限制：课程模拟项目，预算有限，优先使用免费及公开API。

3.3 需求规格

3.3.1 软件系统总体功能/对象结构

系统分为表现层(Vue 3)、业务服务层(Spring Boot)及数据持久层(MySQL/Redis)。

3.3.2 软件子系统功能/对象结构

系统划分为五个核心子系统：大交通票务子系统：机票/车次查询、历史价格走势、候补抢票、优惠券抵扣、订单生命周期管理。
目的地住宿与本地生活子系统：酒店多维搜索、房务库存控制、景点门票预订与订单凭证、五星评价体系（含标签、举报与商家回复）。
AI智能规划与Agent服务子系统：行程JSON生成、AI多轮客服问答、全局通知服务。
旅途社区与用户中心子系统：JWT认证、UGC图文发布、瀑布流社交关系链。
管理后台与可观测性子系统：RBAC权限、操作日志统计大屏、敏感词过滤审核、退款审核 workflow、CSV批量数据导入、用户封禁管理。

3.3.3 描述约定

时间标准：统一采用create_time, update_time字段名。
命名规范：数据库字段采用小写下划线，代码变量采用camelCase。

3.4 CSCI能力需求

3.4.1 AI 行程生成与调度

输入：目的地、天数、预算、人数、偏好标签。
处理：构造Prompt异步调用DeepSeek API，通过JSON Output约束返回结果，并匹配站内Mock资源。
输出：分天展示的JSON行程数据及推荐资源列表。

3.4.2 订单并发与库存控制

行为：用户下单时，Redis优先预减库存，数据库使用原子语句校验，防止超卖。
时限：超时15分钟未支付订单需自动触发库存回滚。

3.4.3 内容审核与社交安全

功能：对UGC内容进行敏感词自动过滤与人工复核，确保社区合规性。

3.5 CSCI外部接口需求

3.5.1 接口标识和接口图

系统与DeepSeek API、第三方地图API及文件存储服务进行交互。

3.5.2 用户接口

风格：卡片式设计，蓝色系主调。组件：Element Plus（统一用于PC Web端）。

3.5.3 硬件接口

运行于标准个人计算机，适配1080P及以上分辨率显示器。

3.5.4 软件接口

DeepSeek API：采用OpenAI兼容协议，Bearer Auth认证。
支付模拟：内部Webhook回调，不接真实网关。

3.5.5 通信接口

协议：HTTP/HTTPS, RESTful API。
数据格式：标准JSON。

3.6 CSCI内部接口需求

子系统C(AI)需调用子系统A/B提供的资源检索接口获取Mock数据。
子系统D生成的内容需推送到子系统E的审核流接口。

3.7 CSCI内部数据需求

核心实体模型(用户、订单、资源、消息、社区内容)需在概要设计阶段统一。

3.8 适应性需求

前端需通过Vue路由懒加载和Vite资源分包适应不同网络环境下的加载速度。

3.9 保密性需求

身份验证：基于JWT的Token验证机制。

权限控制：基于RBAC的角色访问控制，防止越权访问。

3.10 保密性和私密性需求

用户密码存储需进行哈希加密，社区隐私字段在接口层进行数据脱敏。

3.11 CSCI环境需求

开发工具：JDK 21,Node.js,Git,IDE。

部署工具：Nginx配置HTTP Gzip压缩。

3.12 计算机资源需求

3.12.1 计算机硬件需求

本系统(CSCI)主要部署于云端服务器或本地模拟环境，具体硬件需求如下：

处理器(CPU)：建议2核及以上，以支持Java虚拟机的并发处理与大模型API的异步调度。

存储器(RAM)：系统运行总内存需求不低于4GB。其中，Java运行时环境(JRE)建议分配1GB-2GB堆内存，MySQL与Redis实例各需约512MB运行空间。

辅助存储器(硬盘)：初始占用空间约2GB(含系统镜像与基础数据)，建议预留20GB以上固态存储(SSD)空间，用于存储UGC社区图片、系统运行日志及数据库持久化文件。

通信/网络设备：需配备标准的以太网卡或无线网络适配器，支持持续的互联网接入，以确保与外部DeepSeek API及其它第三方地图服务联通。

3.12.2 计算机硬件资源利用需求

为保证系统在演示及测试阶段的稳定性，硬件资源利用限制如下：

处理器利用率：在正常负载下(如单用户行程规划)，平均CPU利用率应低于30%；在模拟并发下单场景下，峰值CPU利用率不应长时间超过70%。

存储器容量利用：系统常驻内存占用应控制在分配总额的80%以内。Redis缓存需配置过期策略，防止Mock数据堆积导致内存溢出。

辅助存储器利用：由于采用图片压缩技术，UGC存储空间增长应可控；系统需配置日志滚动删除策略，确保磁盘占用率不超过90%。

网络带宽利用：单次AI交互产生的数据流量通常在KB级别，峰值带宽占用主要集中在前端静态资源初次加载与社区图片上传阶段。

3.12.3 计算机软件需求

系统开发与运行必须依赖以下软件环境：

操作系统：服务端建议使用Linux(Ubuntu 20.04 LTS 或 CentOS 7.9 及以上)；客户端支持Windows 10/11、macOS 或现代移动端浏览器。

数据库管理系统：MySQL 8.0.x，用于核心业务数据的持久化存储。

缓存软件：Redis 6.2或7.x版本，用于库存预占、会话管理及热点Mock数据缓存。

运行环境与实用软件：

Java Development Kit (JDK) 21：后端服务运行基石。

Nginx 1.20+：负责静态资源托管及后端API的反向代理。

Node.js 18+：前端构建工具Vite的运行环境。

外部API引用：DeepSeek API(兼容 OpenAI 协议版本)，用于支持AI智能规划的核心功能。

3.12.4 计算机通信需求

系统涉及的通信机制与性能指标要求如下：

配置与拓扑结构：采用典型的前后端分离架构。前端(Browser)通过公共网络与后端(Spring Boot Server)连接，后端在内网环境下与MySQL和Redis实例通信。

传输技术与协议：全站基于TCP/IP协议，应用层统一采用HTTP/HTTPS协议及RESTful API规范。

数据传输速率：后端接口响应(不含 AI 生成)时间应在500ms以内；AI行程生成受限于大模型算力，首字响应(Stream)或整体返回时间应控制在3-10秒。

网关需求：需支持跨域资源共享(CORS)配置，以允许前端单页面应用(SPA)的安全访问。

数据的峰值与容量：系统需具备处理短时间内每秒数十次(QPS)请求的能力，特别是针对订票与订房的库存减扣接口。

3.13 软件质量因素

3.13.2 可靠性

数据一致性：在高并发模拟场景下，订票与订房库存不得出现超卖现象，数据库与Redis缓存状态需最终一致。

服务可用性：系统应具备处理大模型API超时或宕机的异常处理机制，保证在非核心功能失效时主流程依然可访问。

3.13.3 可维护性

规范化：代码需遵循阿里巴巴Java开发手册规范，前端遵循Vue 3组合式API开发规范。

模块化：采用前后端分离架构，各功能模块通过RESTful API通信，确保单个模块的修改不影响整体系统。

3.13.4 易用性

交互效率：用户从输入行程需求到生成计划的操作步骤应控制在3步以内。

反馈及时性：所有写操作(如提交订单)需在1秒内给出反馈。AI生成等长耗时操作需提供Loading状态或流式输出。

3.14 设计和实现的约束

技术栈约束：后端必须使用JDK 21与Spring Boot 3.x；前端使用Vue 3 (Vite + JavaScript)；数据库使用MySQL 8.0与Redis。

接口规范：AI模块必须兼容OpenAI API协议，采用异步调用机制以防止阻塞主线程。

数据标准：所有地理位置信息需符合标准经纬度格式，订单金额统一使用分(Integer 类型)存储。

安全性约束：用户密码必须经过BCrypt或类似的单向哈希加密存储，敏感接口需携带JWT令牌验证。

3.15 数据

输入数据：用户注册信息、行程偏好(地点、预算、天数等)、订单支付指令、UGC社区图文内容。
输出数据：AI生成的结构化行程JSON、订单状态凭证、可视化系统监控报表、社区动态瀑布流。
处理量与数据量：系统应支持至少100个模拟并发连接；数据库设计需支持万级订单量与十万级Mock资源数据的快速检索。

3.16 操作

常规操作：用户通过浏览器访问平台，进行登录、搜索资源、AI规划行程、下单及社区发布。
初始化操作：通过执行init.sql脚本完成数据库表结构与基础Mock数据的装载；通过Nginx配置文件完成前端静态资源与后端API反向代理的映射。
恢复操作：若系统宕机，需支持通过重启JAR包及Redis实例实现快速服务恢复；数据库需支持基于Binlog的增量恢复。

3.17 故障处理

API 故障：当DeepSeek API响应超时或由于额度不足返回错误时，系统需拦截错误并返回预定义的“推荐行程模版”，确保用户不看到空白页面。
库存冲突：当多用户同时抢购最后一份资源时，系统应捕获数据库OptimisticLockingFailureException，并向用户提示“库存已售罄”。
错误信息定义：
ERR_001: AI服务暂不可用，请稍后再试。
ERR_002: 库存不足，订单关闭。
ERR_003: 登录过期，请重新认证。

3.18 算法说明

3.18.1 AI 结构化生成算法

概况：利用Prompt Engineering将非结构化的用户需求转化为结构化的JSON数据。
实现细节：系统构建一个包含出行背景的System Prompt，要求模型按预定义的分天行程Schema输出（包含：活动名称、时间点、地点、费用估算）；AI响应落库后支持历史记录查询；API超时时自动降级返回预设模板行程。

3.18.2 库存防超卖算法

概况：基于Redis原子操作与Lua脚本的分布式限流与扣减算法。
公式：IF (Redis.get(stock_key) >= req_num) THEN Redis.decrby(stock_key, req_num) ELSE RETURN -1。

3.19 有关人员需求

普通用户：需具备基础的Web浏览器操作能力，无需专业背景。
系统管理员(开发者)：需熟悉Linux基础命令、Git版本控制以及Spring Boot监控端点(Actuator)的使用。
并发需求：系统需支持50-100名用户同时在线进行行程查询。

3.20 有关培训需求

系统应交付《用户操作指南》，涵盖如何启动AI行程助手以及如何进行订单核销。
开发者需提供《部署维护文档》，说明环境搭建、API Key配置及数据库迁移步骤。

3.21 有关后勤需求

维护支持：采用GitHub/GitLab进行代码版本管理。
托管环境：可运行于配备4G内存以上的轻量级云服务器或本地实验环境计算机。

3.22 其他需求

可观测性：系统管理端需集成轻量级监控大屏，实时展示系统QPS、AI响应耗时及错误日志占比。

3.23 包装需求

交付物组成：源代码压缩包、数据库初始化.sql文件、README.md(包含运行步骤)、演示视频(MP4 格式)。
标签规范：代码库需打上v1.0-final的Release标签。

3.24 需求的优先次序和关键程度

关键(Essential)：用户认证、AI行程生成主流程、订单支付逻辑、库存安全校验。
重要(Important)：社区图文上传、后台资源管理模块、系统监控报警。
重要(Important)：历史价格趋势 ECharts 图表展示、CSV 批量数据导入、候补抢票系统。
可选(Optional)：多语言切换、AI语音输入交互。

4 合格性规定

需求项	需求说明	合格性方法
功能需求	实现 AI 行程规划与订单闭环	演示：展示从输入到生成再到下单的全流程
并发性能	支持库存不超卖	测试：使用JMeter进行50线程并发下单压力测试
接口规范	RESTful API 与 JSON 数据标准	审查：检查Knife4j接口文档与代码实现
安全性	密码加密与 JWT 鉴权	分析：检查数据库存储内容与拦截器逻辑

5 需求可追踪性

本说明书中的所有CSCI需求均可追溯至《TravelMate软件开发计划书》中定义的业务蓝图。
功能需求细项对应《详细设计说明书》中的模块设计章节。

6 尚未解决的问题

外部API稳定性：目前依赖的第三方DeepSeek API存在突发性响应延迟，未来考虑引入本地模型或缓存池作为备选。
支付接口集成：当前为模拟支付，若需上线需重新对接真实支付网关(如支付宝、微信支付)。

7 注解

CSCI: Computer Software Configuration Item，本说明书指代“伴游”系统整体。
UGC: User Generated Content，用户生成的社区内容。
JWT: JSON Web Token，用于身份认证的开放标准。

附录

无